

Báo cáo rà soát test code

<!-- framework: pytest | run_id: run_20260617_212152_611966 -->

Framework: pytest | Run ID: run_20260617_212152_611966

Tóm tắt kỹ thuật

- Framework: pytest
- Kết quả chạy: 9 passed, coverage 100%
- Loại vấn đề execution: Không có lỗi thực thi, các test đều chạy thành công.
- Vấn đề chính ngăn cản: Các test được sinh ra không liên quan tới yêu cầu và kế hoạch kiểm thử đã cho (yêu cầu về các helper make* của pytester, tạo file cấu hình, ghi binary, v.v.). Do đó, mặc dù test chạy thành công, chúng không đáp ứng các tiêu chí chức năng, đầu vào/đầu ra, và các trường hợp kiểm thử được liệt kê trong Test Plan.
- Artifact đã review: test_generated.py (test tự động sinh) so với source_under_test.py (mã nguồn math_rules.py).

Lỗi nghiêm trọng

- Bằng chứng:
- Yêu cầu (Requirement JSON) yêu cầu kiểm thử các hàm helper makepyfile, maketxtfile, makeconftest, make_tox_ini, make_pyproject_toml, và ghi binary bằng Path.write_bytes.
- Test plan (Test Plan JSON) liệt kê các TC-001 ~ TC-013 liên quan tới việc tạo file cấu hình, file Python, file text, và file binary.
- Mã test sinh ra chỉ kiểm thử ba hàm factorial, clamp, safe_divide trong source_under_test.py và không có bất kỳ lời gọi nào tới pytester hay các helper trên.
- Ảnh hưởng: Các test không kiểm chứng bất kỳ yêu cầu nào, do đó không thể khẳng định tính đúng đắn của module pytester_file_helpers. Đây là lỗi nghiêm trọng khiến bộ kiểm thử vô hiệu.
- Cách sửa: Thay thế nội dung test hiện tại bằng các test thực hiện các hành động được mô tả trong Test Plan, ví dụ:
def test_make_conftest(pytester):
 path = pytester.makeconftest("def test_dummy(): assert True")
 assert path.name == "conftest.py"
 assert path.read_text() == "def test_dummy(): assert True"
Tương tự cho make_tox_ini, make_pyproject_toml, makepyfile, maketxtfile, và ghi binary với Path.write_bytes.

Test còn thiếu

- Bằng chứng:
- Các TC-001 ~ TC-013 (positive, negative, boundary, clarification) không được hiện thực trong test_generated.py.
- Ảnh hưởng: Thiếu kiểm thử cho:
- Tạo file cấu hình (conftest.py, tox.ini, pytest.toml, pyproject.toml).
- Tạo nhiều file Python/text trong một lần gọi.
- Ghi dữ liệu binary và kiểm tra nội dung.
- Các trường hợp lỗi (TypeError, ValueError) khi truyền tham số không hợp lệ.
- Kiểm thử giới hạn kích thước file (1 MB).

- Kiểm thử tài liệu/định vị file (TC-012, TC-013).
- Cách sửa: Thêm các test tương ứng với từng TC trong Test Plan, sử dụng fixture pytester và xác thực nội dung file như yêu cầu.

Assertion yếu

- Bằng chứng:
- Các test hiện tại chỉ có các assert đơn giản (`assert factorial(5) == 120`, `assert clamp(...) == ...`) và kiểm tra thông điệp lỗi.
- Không có assert nào kiểm chứng đường dẫn trả về của các hàm helper, tồn tại file, hoặc nội dung binary.
- Ảnh hưởng: Các assert hiện có không đủ để chứng minh rằng các helper thực hiện đúng hành vi ghi file, do đó độ tin cậy của bộ kiểm thử thấp.
- Cách sửa: Thêm assert kiểm tra:
- `path.exists()`
- `path.read_text() == expected` hoặc `path.read_bytes() == expected_bytes`
- Kiểm tra tên file và vị trí trong thư mục tạm của pytester.

Rủi ro maintainability

- Bằng chứng:
- Test hiện tại phụ thuộc vào các hàm trong `source_under_test.py` (`factorial`, `clamp`, `safe_divide`) mà không liên quan tới module được yêu cầu.
- Khi yêu cầu thay đổi (thêm các helper), các test này sẽ không cần sửa, dẫn tới độ phủ không phản ánh thực tế.
- Ảnh hưởng: Dễ gây nhầm lẫn cho người bảo trì, vì bộ test không phản ánh mục tiêu dự án và có thể bị bỏ qua khi cập nhật tính năng.
- Cách sửa: Đảm bảo test luôn gắn liền với các hàm/đối tượng thực tế cần kiểm thử; loại bỏ các test không liên quan hoặc di chuyển chúng vào file test riêng cho `math_rules.py` nếu vẫn muốn giữ.

Gợi ý sửa ngay

1. Xóa hoặc tách riêng các test hiện tại (`factorial`, `clamp`, `safe_divide`) ra một file test riêng nếu chúng không thuộc phạm vi yêu cầu.

2. Thêm các test cho các helper `make*` theo đúng Test Plan, ví dụ:

```
def test_makepyfile_multiple(pytester):
    path = pytester.makepyfile(module1="print('one')", module2="print('two')")
    assert (path / "module1.py").read_text() == "print('one')"
```

3. Kiểm tra lỗi: viết test cho trường hợp truyền kiểu sai (non-string, non-Path) và xác nhận `TypeError` được raise.

4. Kiểm thử binary:

```
def test_write_bytes(tmp_path):
    file = tmp_path / "binary.bin"
    data = bytes([0, 255, 127, 0])
    file.write_bytes(data)
    assert file.read_bytes() == data
```

5. Thêm test boundary cho file 1 MB và xác nhận không gây lỗi thời gian hoặc bộ nhớ.
6. Cập nhật tài liệu (TC-012, TC-013) bằng cách trích xuất chữ ký thực tế của các hàm helper từ source (nếu có) hoặc ghi chú trong comment.

Áp dụng các bước trên sẽ đưa bộ kiểm thử phù hợp với yêu cầu, tăng độ bao phủ thực tế và giảm rủi ro bảo trì.